

Decision Tree Models

Dr. Marcelo Feighelstein

Agenda

01

Why Decision Trees?

Intuition, terminology, and recursive splitting

02

What is a Split?

Features, thresholds, Gini impurity, and finding the best split

ĆĊ

Building the Model

Dataset preparation, Python/sklearn, full decision tree

ĆÇ

Making Predictions

Tracing a patient through the tree node by node

05

Model Evaluation & Overfitting

Train vs. test performance, confusion matrices, pruning with max_depth

ĆĎ

Hyperparameters

max_depth, min_samples_leaf, class_weight and more

07

Feature Importance

How the tree explains itself, calculation example, limits

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
2	40	M	ATA	140	289	0	Normal	172	N	0	Up	0
3	49	F	NAP	160	180	0	Normal	156	N	1	Flat	1
4	37	M	ATA	130	283	0	ST	98	N	0	Up	0
5	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
6	54	M	NAP	150	195	0	Normal	122	N	0	Up	0
7	39	M	NAP	120	339	0	Normal	170	N	0	Up	0
8	45	F	ATA	130	237	0	Normal	170	N	0	Up	0

Dataset: Heart Failure Prediction

Target: HeartDisease — 0 = No Disease, 1 = Disease

Demographics

Age, Sex

Symptoms

ChestPainType, ExerciseAngina

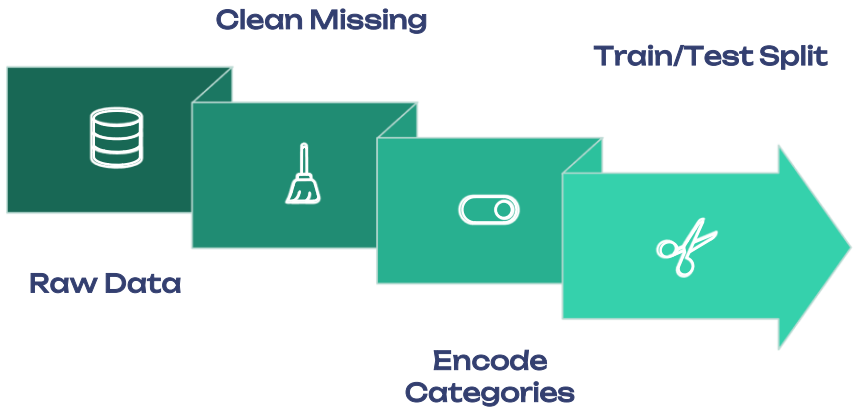
Vitals

RestingBP, MaxHR, Oldpeak

Lab / ECG

Cholesterol, FastingBS, RestingECG, ST_Slope

Data Preparation



Unlike KNN, Decision Trees do **not** require feature normalization — but sklearn still requires all inputs to be **numeric**.

Key Preparation Steps

1 Check Missing Values

Handle nulls before training

2 Binary Encoding

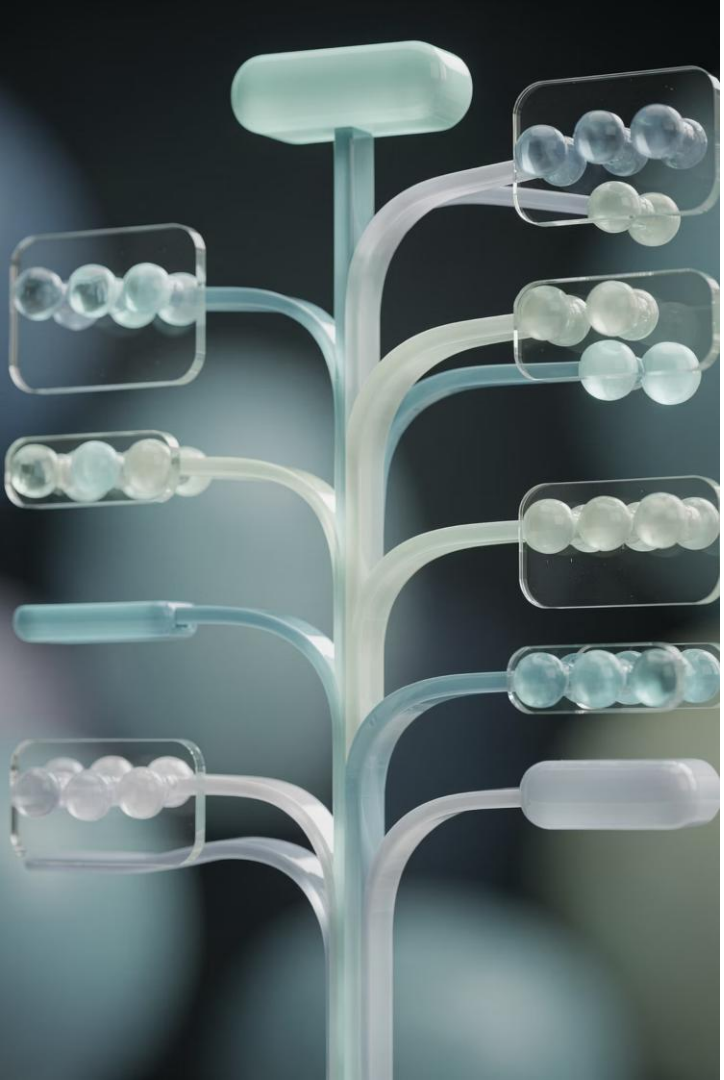
Convert Yes/No columns to 0 and 1

3 One-Hot Encoding

Convert multi-class columns (e.g. ChestPainType) using pd.get_dummies()

4 Train/Test Split

70% train, 30% test — use random_state=42 for reproducibility

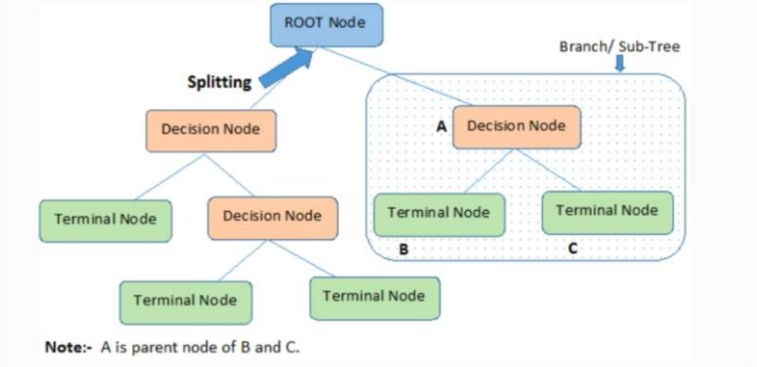


Why Decision Trees?

Decision Trees mirror how humans naturally make decisions.

- Easy to understand and visualize
- Works for both classification and regression tasks
- No need for feature scaling or normalization
- Handles numeric and categorical data
- Logic is transparent and easy to explain to non-experts

Tree Terminology



Source: Kdnuggets

Root Node

The **first decision point** — uses the single most informative feature

Decision Node

An internal node where **another split** is made on a feature

Branch

The connection between nodes — represents a **true/false path**

Leaf / Terminal Node

No further splits — **outputs the final prediction** by majority vote

Pruning

Limiting or removing parts of the tree to **reduce overfitting**

What is a Split?

A split selects one feature and one threshold value, then divides the dataset into two groups: samples that meet the condition go left (True), the rest go right (False).

01

Pick a Feature

Age, MaxHR, ST_Slope_Up

02

Choose a Threshold

Age < 55, MaxHR < 150, ST_Slope_Up ≤ 0.5

03

Divide the Data

Left branch: condition is True.
Right branch: condition is False.

🔢 Numeric Feature Splits

- Age < 55
- MaxHR < 150
- Oldpeak < 1.5

🏷️ Categorical Feature Splits

- ST_Slope = 'Up'
- ChestPainType = 'ASY'
- Sex = 'M'

CHAPTER 1

The Basic Idea: Recursive Splitting

01

Choose Split

Select feature and threshold

02

Partition Data

Split into two groups

03

Recurse

Repeat for each subgroup

04

Stop

Predict or limit tree

> The tree grows by asking: *"What one question best organizes the data right now?"*

At every step, the algorithm selects the **single best rule** — one feature, one threshold — that most cleanly separates the classes.

Key Insight

The process is **recursive**: each subgroup is treated as a new problem and split again, forming a tree structure from root to leaves.

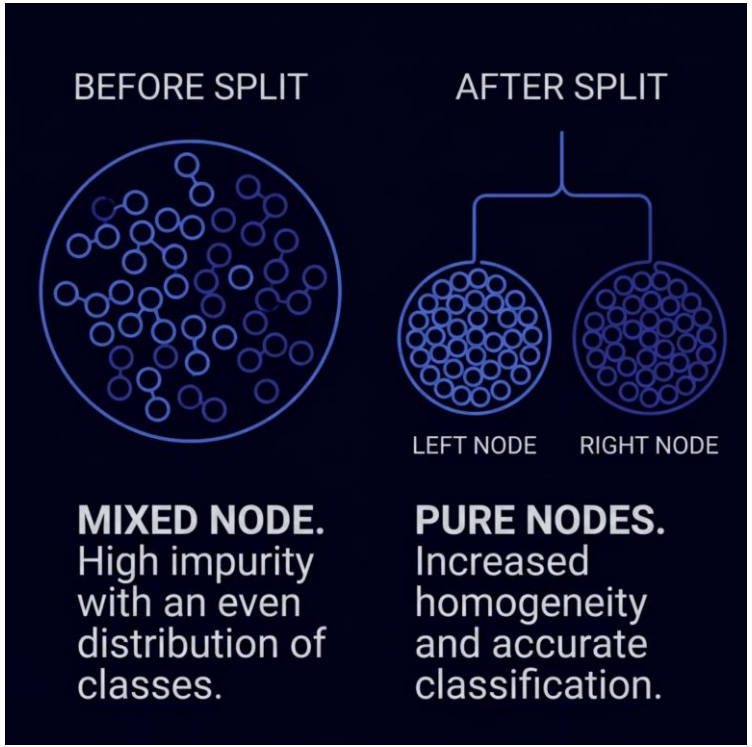
- Splits are **binary**: left branch vs. right branch
- Splitting continues until a stopping criterion is met
- Each leaf makes a **majority-vote prediction**

What Makes a Good Split?

Goal: Homogeneous Subgroups

A **pure leaf** contains samples from only one class. A good split moves us closer to purity on both sides.

Node	Class 0	Class 1	Purity
Before split	50%	50%	❌ Mixed
Left (after)	95%	5%	✅ High
Right (after)	10%	90%	✅ High



CHAPTER 2

Gini Impurity

Intuition

Gini measures **how mixed the classes are** inside a node. Lower Gini = more pure = better.

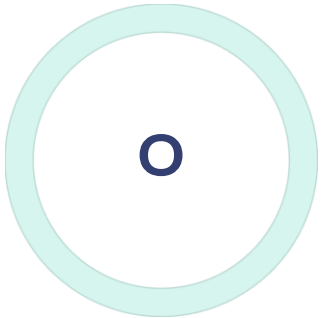
$$Gini = 1 - \sum_{k} p_k^2$$

Where *p_k* is the proportion of class *k* in the node.

Worked Example

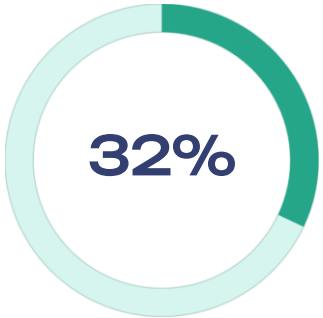
A node with 10 samples: **8 class 0, 2 class 1**

$$Gini = 1 - (0.8^2 + 0.2^2) = 1 - 0.68 = 0.32$$



Pure Node

All samples same class — perfect split



Mostly One Class

8 of 10 samples are class 0 — good split



Maximally Mixed

50/50 split — worst possible node

How to Find the Best Split

The goal is to find the feature and threshold that best separate the classes, maximizing the reduction in impurity.

01

Iterate Features

Go through every input feature available in the dataset.

ĆČ

Test Thresholds

For each feature, evaluate every possible split point (threshold) to divide the data.

ĆĊ

Calculate Gain

For each potential split, measure the reduction in Gini impurity (or other metric) for the resulting child nodes.

04

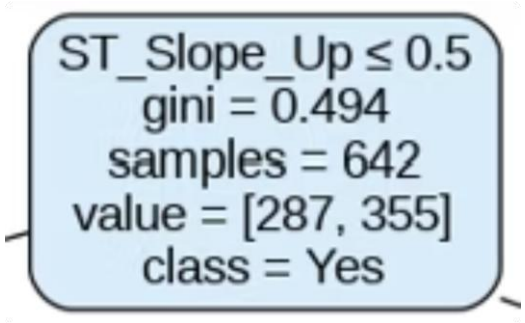
Select Best

Choose the feature and threshold combination that yields the greatest impurity reduction.

This exhaustive search guarantees finding the locally optimal split at each decision node.

ROOT SPLIT DECISION TREE

Selecting the Root: ST_Slope_Up as a Feature



Rule: ST_Slope_Up ≤ 0.5?

The model evaluates **every possible feature and threshold**, then selects the one that best separates heart disease cases from healthy patients.

ST_Slope_Up is a binary-like feature (0 or 1) indicating whether the ST segment slope is upward. The threshold **0.5** effectively splits the data into up vs. not-up.

Gini Calculation at the Root Node

Root Node

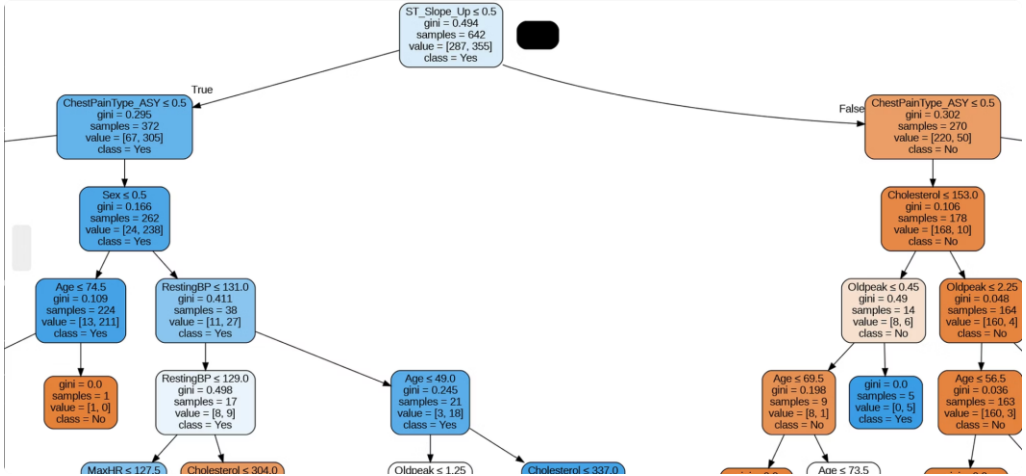
samples = 642, class distribution: [287 No, 355 Yes]

Gini = $1 - (287/642)^2 - (355/642)^2 = 1 - 0.200 - 0.306 = 0.494$

The root node is chosen because ST_Slope_Up ≤ 0.5 produces the largest impurity decrease (0.196) among all possible features and thresholds.

Anatomy of a Decision Node

Let's zoom into a single decision point within the tree to understand how classifications are made at each step.



Feature Test

Each node evaluates a specific feature from the dataset, like 'Age' or 'Cholesterol'.

Threshold Value

A precise value (e.g., 56 years, 120 mg/dL) acts as the cutoff for the split.

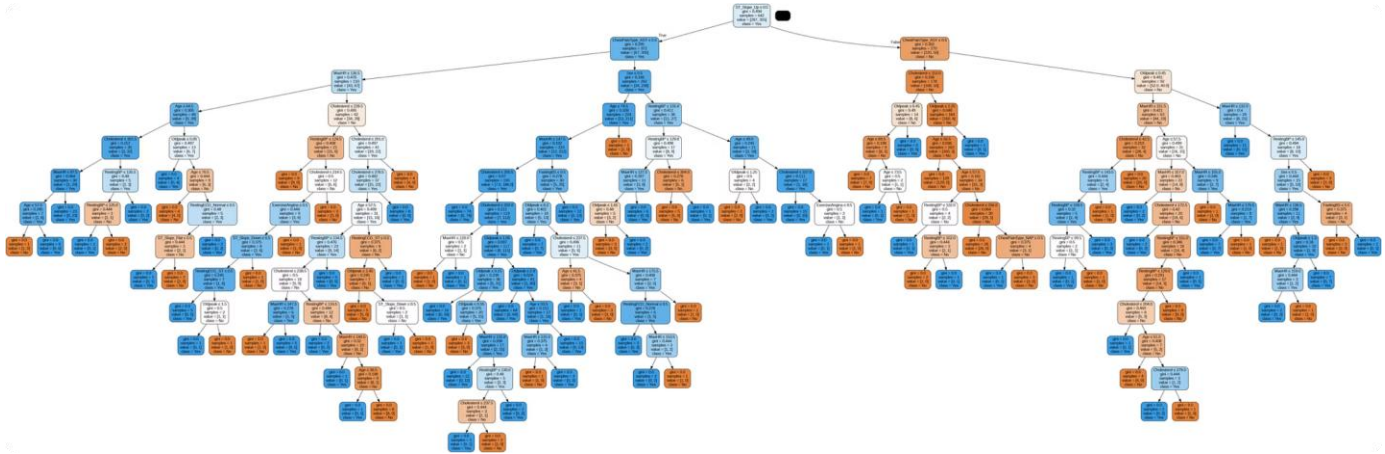
Binary Split

The data is divided into two distinct groups based on whether they meet the condition (true/false).

Purity Objective

The primary goal of every split is to reduce impurity in the resulting child nodes, making them more homogeneous.

Full Decision Tree



DECISION TREES | IMPURITY

Entropy and Information Gain

Gini vs. Entropy: Both work well in practice. Gini is the sklearn default and slightly faster to compute.

Entropy: Another Impurity Measure

$$H = -\sum_{k} p_k \log_2(p_k)$$

A pure node has **entropy = 0**. A fully mixed node has entropy = 1. Like Gini, lower is better after a split.

Information Gain

$$IG = H_{parent} - \frac{N_L}{N} H_{left} - \frac{N_R}{N} H_{right}$$

We **gain information** when the split creates more ordered, homogeneous groups. The tree always selects the split with the highest gain.

Building a Decision Tree in Python

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import
train_test_split
from sklearn.metrics import (accuracy_score,
                             recall_score,
                             precision_score)

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.3, random_state=42
)

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)
print(accuracy_score(y_test, y_pred))
```

What Each Line Does

→ train_test_split()

Randomly divides data into training and held-out test sets

→ dt.fit()

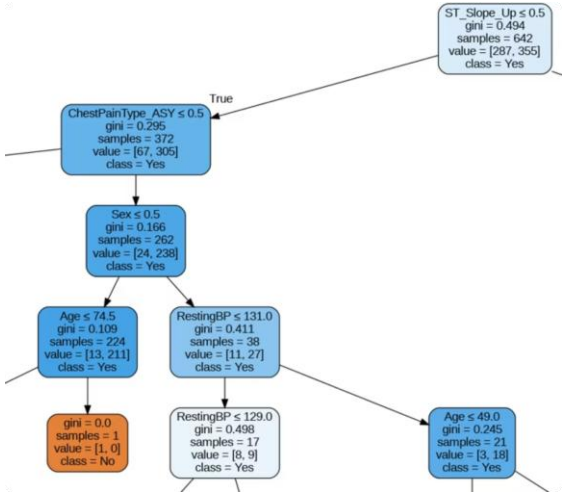
Recursively learns feature-threshold splits from training data

→ dt.predict()

Routes each test sample down the learned tree to a leaf prediction

No normalization needed — Decision Trees are scale-invariant by design.

How Does a Tree Make a Prediction?



Tracing One Patient Through the Tree

1 — Root Node

Check: ST_Slope_Up ≤ 0.5? → Yes, go left

2 — Decision Node 1

Check: ChestPainType_ASY ≤ 0.5? → No, go right

3 — Decision Node 2

Check: Sex ≤ 0.5? → Yes, go left

4 — Decision Node 3

Check: Age ≤ 74.5 → No, go right

5 — Leaf Node

Samples: [1, 0] → Majority is class 0

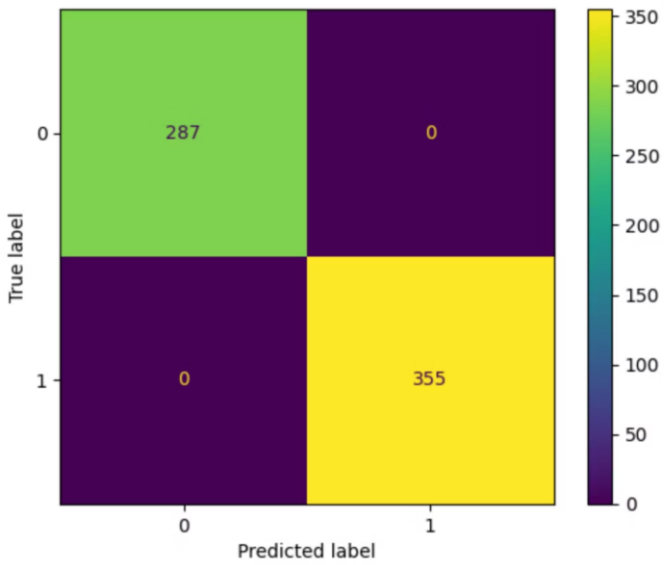
6 — Prediction

HeartDisease = No (class 0)

ⓘ Each leaf predicts by **majority vote**: whichever class has the most training samples in that leaf wins.

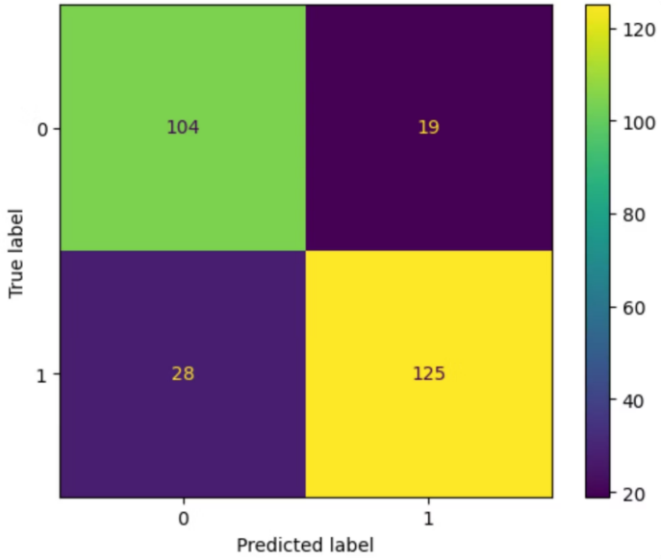
Model Evaluation: Test Performance

Training Set



Accuracy: 100% | Recall: 100% | Precision: 100%

Test Set



Accuracy: 82% | Recall: 80% | Precision: 87%

> **Is this the best decision tree we can build for the given train/test split?**

Train vs. Test Performance: Overfitting Signal

Training Set

100%

Accuracy · Recall · Precision

The tree has perfectly memorized every training example — it has a rule for each one.

Test Set

82%

Accuracy on unseen data

The gap between train and test performance is the key signal. **High training performance does not mean a good model.**

⚠ ⚠️ Train accuracy = 100% while test accuracy = 82% strongly suggests **overfitting**.

Overfitting: Memorizing vs. Learning

A very deep tree creates many highly specific rules — perfectly fitting training data but **failing to generalize** to new patients.

Underfitting

Tree too shallow — misses real patterns. High bias, low variance.

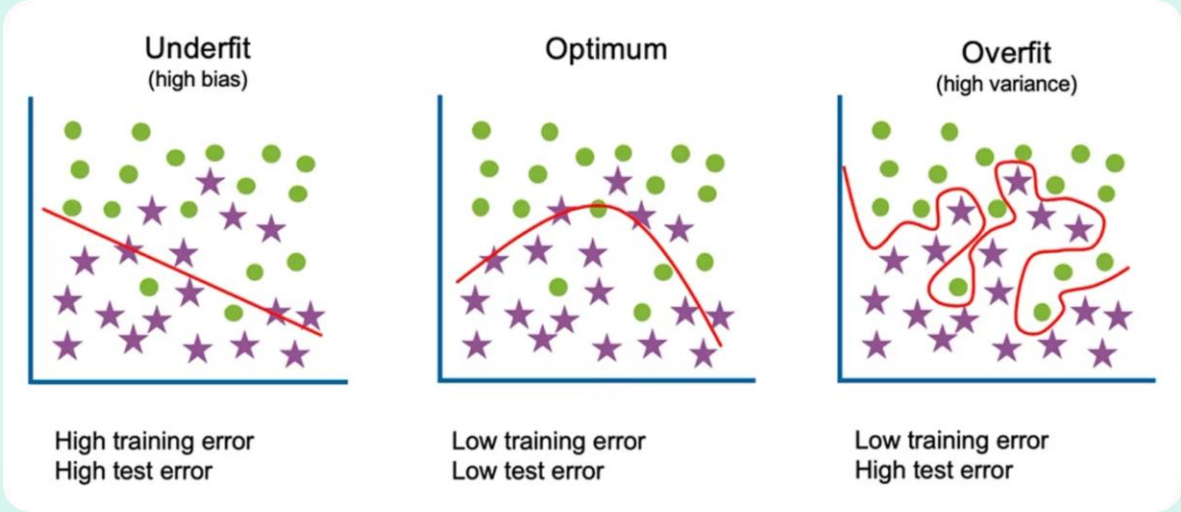
Good Fit

Balanced depth — captures real structure without memorizing noise.

Overfitting

Tree too deep — memorizes training quirks. Low bias, high variance.

Generalization is the ability to derive good rules that apply to unseen data — the true goal of any model.



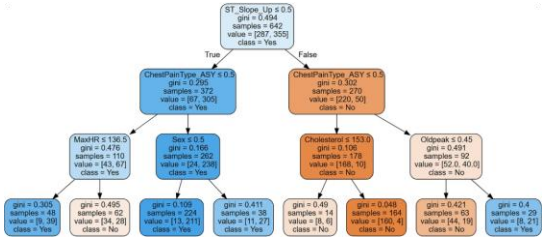
CHAPTER 5

Reducing Overfitting: Pruning with max_depth

Full Unpruned Tree

Deep, many nodes

Test accuracy: 82.25%



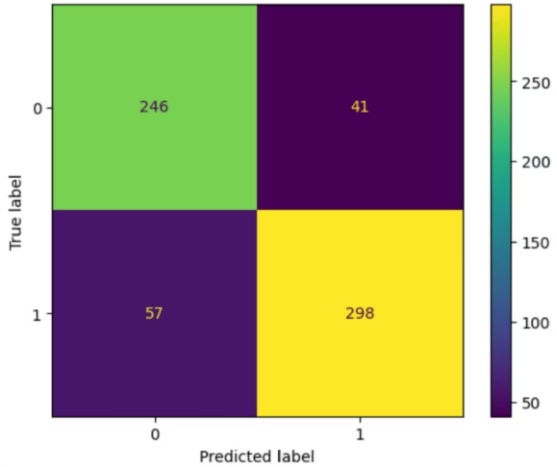
Pruned Tree (max_depth=3)

Shallow — 3 levels

Test accuracy: 84.78%

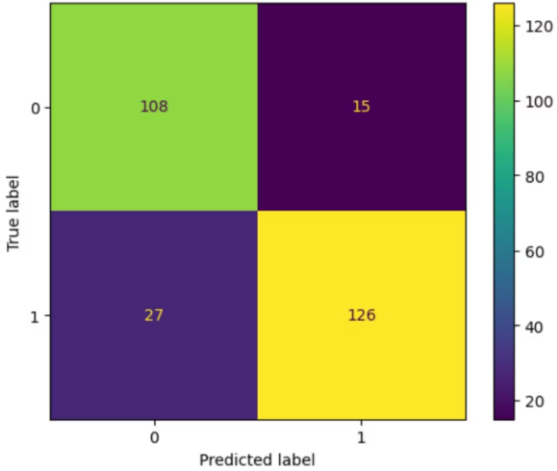
A simpler tree can outperform a complex one on test data by avoiding overfitting noise in the training set.

Training Set



Accuracy: 85% | Recall: 84% | Precision: 88%

Test Set



Accuracy: 85% | Recall: 82% | Precision: 89%

Train vs. Test Gap: Overfitting Indicator

The smaller the gap between training and test accuracy, the less the model is overfitting — and the better it generalizes.

Metric	Full Tree (Train)	Full Tree (Test)	Gap	Pruned Tree (Train)	Pruned Tree (Test)	Gap
Accuracy	100%	82%	18%	85%	85%	0%
Recall	100%	80%	20%	84%	82%	2%
Precision	100%	87%	13%	88%	89%	-1%

Large Gap (Full Tree)

Train 100%, Test 82% → High overfitting. The model memorized training data.

Small Gap (Pruned Tree)

Train 85%, Test 85% → Low overfitting. The model generalizes well.

Key Takeaway

Minimizing the train-test gap is the real goal. A perfect training score is a red flag, not a success.

> A simpler model, by focusing on robust patterns and ignoring noise, often generalizes better to unseen data.

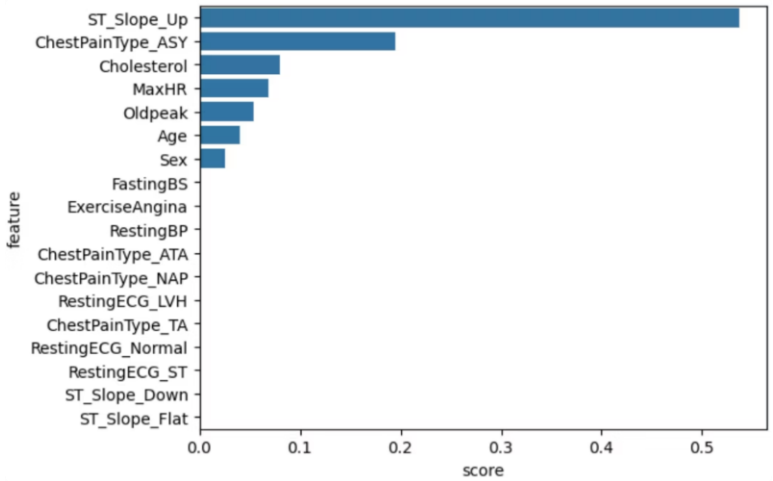
Full Hyperparameter Reference

Parameter	What It Controls	Practical Guidance
`max_depth`	Maximum depth of the tree	Start at 3–7; deep trees overfit
`min_samples_leaf`	Min samples required in a leaf	Larger = smoother; ≥5 for regression
`min_samples_split`	Min samples needed to split a node	Prevents splits on tiny subgroups
`criterion`	Impurity measure: gini or entropy	Gini is default; entropy sometimes helps
`class_weight`	Weight classes during training	Use 'balanced' for imbalanced datasets
`sample_weight`	Weight individual training samples	Useful for cost-sensitive learning

Never choose hyperparameters by peeking at the test set. Use **cross-validation on training data**, then evaluate once on the test set.

CHAPTER 6

Feature Importance: How the Tree Explains Itself



What Is Feature Importance?

Decision Trees can estimate which features contributed most to predictions — a form of **global model explanation**.

Importance is **global**: it summarizes how useful each feature was across the *entire tree*.

High Importance

ST_Slope_Up — used at the root, dramatically reduces Gini across many samples

Medium Importance

MaxHR — used frequently in mid-level nodes with moderate impurity reduction

Low Importance

Age or RestingBS — used only in a small lower node affecting few samples

All feature importance scores sum to 1. This normalization makes them easy to compare — a score of 0.45 means that feature was responsible for 45% of the total impurity reduction across the tree.

How Feature Importance Is Calculated

The Formula

$$\Delta i = \frac{N_{node}}{N_{total}} \left[Gini_{parent} - \frac{N_L}{N_{node}} Gini_L - \frac{N_R}{N_{node}} Gini_R \right]$$

Each feature's importance = **sum of impurity decreases** across all splits where it was used, normalized to sum to 1.

Worked Example

Node	Gini	Samples
Parent	0.50	100
Left child	0.10	40
Right child	0.20	60

$$\Delta i = 0.50 - (0.4 \times 0.10 + 0.6 \times 0.20) = 0.50 - 0.16 = 0.34$$

Step-by-Step Process

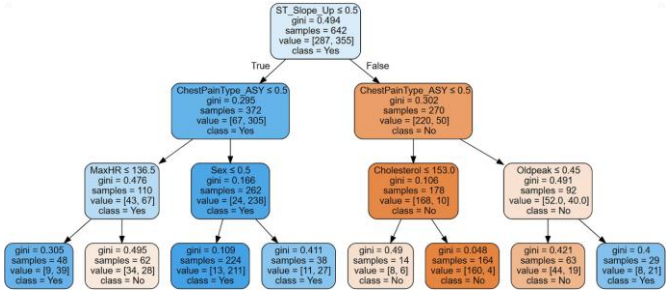
1. **Compute impurity before the split**
2. **Compute weighted impurity of children**
3. **Subtract to get impurity decrease**
4. **Assign decrease to the feature used**
5. **Sum over all splits, then normalize to 1**

Features used **near the root** with **many samples** receive the highest importance scores.

Feature Importance: ST_Slope_Up Example

Using the pruned tree (max_depth=3), we calculate the impurity decrease at the root node where ST_Slope_Up ≤ 0.5 is used as the split.

Root Node Split



ST_Slope_Up ≤ 0.5 is the root — the most impactful split in the tree.

Step-by-Step Calculation

1. Parent node: gini = 0.494, samples = 642
2. Left child (True): gini = 0.295, samples = 372
3. Right child (False): gini = 0.302, samples = 270
4. $\Delta I = 0.494 - (372/642 \times 0.295) - (270/642 \times 0.302) = 0.494 - 0.171 - 0.127 = 0.196$
5. This value is then divided by the sum of all impurity decreases across the tree and normalized to sum to 1.

ST_Slope_Up achieves the highest single impurity decrease (0.196) in the tree — confirming it as the most important feature. After normalization across all splits, it accounts for the largest share of the total importance score.

Key Takeaways

A summary of the core concepts covered in this lecture.

🌳 Decision Trees

Split data recursively using feature thresholds. Each path from root to leaf is a decision rule.

📐 Gini Impurity

Measures how mixed a node is. The best split minimizes weighted Gini across both child nodes.

🏗️ Building with sklearn

Use DecisionTreeClassifier. Categorical features must be encoded numerically before training.

⚠️ Overfitting

A perfect training score (100%) is a red flag. A large train-test gap means the model memorized, not learned.

✂️ Pruning

Limiting max_depth reduces overfitting. A simpler tree (depth=3) can outperform a full tree on test data.

📊 Feature Importance

Scores are based on impurity decrease, normalized to sum to 1. ST_Slope_Up is the most important feature in our model.

> The goal is not a perfect training score — it's a model that generalizes.

Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Use chatbots to help you code and explore — but guide them intelligently and always judge their results critically.

📧 Email

MarcelFei@telhai.ac.il

🕐 Office Hours

Per Request

📍 Location

Building 6, Room 309, Eastern Campus